Ders 3 Genişletilmiş Notu: MapReduce İyileştirmeleri, Shingling ve MinHashing

Ders 3 Genişletilmiş Notu: MapReduce İyileştirmeleri, Shingling ve MinHashing

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. MapReduce Programlama Modelinin Tekrarı
- 2. MapReduce'un Arka Planda Yaptığı İşler
- 3. Büyük Veri Yazılım Yığını
- 4. Ana Düğüm ve Arıza Yönetimi
- 5. İyileştirme 1: Birleştirici (Combiner) Fonksiyonu
- 6. Birleştiricinin Şartı: Değişme ve Birleşme Özellikleri
- 7. İyileştirme 2: Bölümleme (Partition) Fonksiyonu
- 8. MapReduce Gerçeklemeleri ve Bulut Ortamları
- 9. Örnek Soru: Asal Bölenlerle MapReduce
- 10. Benzer Öğeleri Bulma Problemi ve Uygulamaları
- 11. Üç Adım: Shingling, MinHashing, LSH
- 12. Shingling
- 13. Jaccard Benzerliği
- 14. Belgeleri 0/1 Kolon Vektörü Olarak Göstermek
- 15. MinHashing ve İmza Matrisi
- 16. İmza Benzerliği ile Jaccard Benzerliğinin Karşılaştırılması
- 17. Soru-Cevap: Girdi Matrisinden İmza Matrisine Geçiş

1. MapReduce Programlama Modelinin Tekrarı

Ders online yapıldı. Hoca derse, önceki derste kalınan yeri hatırlatarak başladı. Önce MapReduce programlama modelinin (MapReduce Programming Model) kısa bir tekrarını yapacağını, sonra derse devam edeceğini söyledi. Anlatımın ilk yarısı İngilizce, ikinci yarısının büyük kısmı Türkçe yapıldı.

> **Hocanın vurgusu:** Bu ders önemli, çünkü dönem projesi tamamen MapReduce programlama modeli üzerine olacak.

1.1 Girdi Çiftleri ve Veri Parçaları

MapReduce'ta başlangıçta girdi olarak anahtar-değer çiftleri (key-value pairs) vardır. Burada anahtar bir veri parçasının (data chunk) kimliğidir, değer ise genelde veri parçasının kendisidir. Hoca bunu dağıtık dosya sistemiyle ilişkilendirdi. Bir dosyayı dağıtık dosya sistemine koyduğumuzda dosya parçalara bölünür ve bu parçalar sunuculara dağıtılır. Slayttaki dikdörtgenlerin her biri bir sunucuydu ve içlerindeki kutular büyük bir dosyanın veri parçalarıydı. Her parçanın onu tanımlayan bir kimliği vardır.

1.2 Map, Gruplama ve Reduce

Map fonksiyonunu geliştirici yazar. MapReduce programlama kütüphanesi bu fonksiyonu alır ve her veri parçasına uygular. Map fonksiyonu, veriden problem açısından önemli olan şeyi çıkarır.

Kelime sayma (word count) örneğinde veri parçası bir metin parçasıdır. Map fonksiyonu bu metindeki kelimeleri tek tek çıkarır ve her kelime için bir ara anahtar-değer çifti (intermediary key-value pair) üretir. Anahtar kelimenin kendisi, değer ise 1'dir. Çünkü metinde görülen her kelime bir kez sayılır.

Ardından kütüphane bu ara çiftleri anahtara göre gruplar (group by key). Hocanın slayttaki örneğinde bir Map işinde "the" ve "crew" kelimeleri çıkarılmıştı. Başka bir Map işi de "crew" kelimesini çıkarmıştı. Gruplama adımı, farklı Map işlerinden gelen aynı anahtarlı çiftleri bir araya getirir. Bu işi programcı değil, MapReduce kütüphanesinin kendisi yapar.

Son adım Reduce fonksiyonudur. Bunu da programcı yazar. Kelime sayma örneğinde Reduce, her anahtar için değerleri toplar. Örneğin bir anahtarın değerleri 1, 1, 1 ise Reduce sonucu 3 olur.

Özetle kelime sayma örneğinde:

- **Map:** Girdi veri parçasından kelimeleri çıkarır, her biri için (kelime, 1) üretir.
- **Gruplama:** Ara çiftleri anahtara göre bir araya getirir. Kütüphane yapar.
- **Reduce:** Aynı anahtara ait değerleri toplar.

1.3 Kod Olarak Map ve Reduce

Hoca, öğrencilere basit bir MapReduce programı yazmaları istense ne yapacaklarını sordu. Önce bir MapReduce kütüphanesi indirilir. Hadoop bunlardan biridir, Spark bir diğeri. Kütüphane, Map ve Reduce fonksiyonlarının arayüzünü, yani imzasını (signature) hazır verir. Programcı yalnız fonksiyonların içini doldurur.

Kelime sayma için Map fonksiyonunda, girdi değeri olan metin parçasındaki her kelime üzerinde bir döngü (for each) kurulur. Her kelime için anahtarı kelime, değeri 1 olan bir ara çift üretilir. Reduce fonksiyonunda ise, gruplanmış her anahtarın değer listesi üzerinde bir döngüyle değerler toplanır. Böylece her kelimenin son sayısı bulunur.

1.4 Daha Önce Görülen Örnekler

Hoca, MapReduce ile şimdiye kadar konuşulan örnekleri hatırlattı:

- Büyük bir metin belgesinde her kelimeyi saymak.
- Büyük bir belgede geçen beş kelimelik dizileri saymak.
- Bir web sitesinde her URL'nin boyutu biliniyorsa, o siteye ait toplam boyutu bulmak.

2. MapReduce'un Arka Planda Yaptığı İşler

Hoca, Map ve Reduce fonksiyonlarının nasıl çalıştırıldığını gösteren şemaya döndü. Her Map fonksiyonu kendi yerel veri parçasını girdi olarak alır ve anahtar-değer çiftleri üretir. Bir veri parçası sunucusunda birden fazla Map fonksiyonu olabilir. Genelde her veri parçasına bir Map fonksiyonu düşer. Slayttaki örnekte bir dosyaya ait 7 veri parçası vardı, bu yüzden 7 farklı Map işi vardı.

Map işleri ara çiftleri ürettikten sonra kütüphanenin bölümleme (partitioning) ve sıralama-gruplama (sort and group) işlevleri devreye girer. Kütüphane hangi anahtarın hangi Reducer'a gideceğini bir hash fonksiyonu uygulayarak belirler. Sonra her Reducer'a gidecek çiftleri içeren küçük bir dosya oluşturur. Bu dosya dağıtık depolamaya değil, sunucunun **yerel** deposuna yazılır.

Ardından süreç şöyle ilerler:

- Map tarafındaki kütüphane ad sunucusuna (name server) sinyal gönderir: "Ara çiftleri hazırladım, ilgili Reducer'a haber ver." Hoca ad sunucusunun şemada çizilmediğini, ama orada bir yerde olduğunu düşünmemizi söyledi.

- Ad sunucusu ilgili Reduce sunucusuna sinyal gönderir.
- Reduce sunucusundaki kütüphane gidip bu dosyayı kendi yerel deposuna kopyalar.
- Kütüphane çiftleri anahtara göre gruplar.
- Reducer, programcının yazdığı işlemi (örneğin toplama) uygular ve sonucu yazar.

Ara çiftlerin Map sunucusundan Reduce sunucusuna taşınması tamamen kütüphanenin sorumluluğundadır. Geliştirici bunun için kod yazmaz. Hoca ayrıca genelde Reduce işi sayısının Map işi sayısından az olduğunu belirtti.

2.1 Kütüphanenin Sorumlulukları

Hoca, programcının değil ortamın (kütüphanenin) üstlendiği işleri tek tek saydı:

- **Girdinin bölünmesi:** Dosyayı veri parçalarına bölmek dağıtık dosya sisteminin, yani ortamın işidir.
- **Zamanlama:** Programın makineler üzerinde çalıştırılmasını zamanlamak. Önce Map işleri çalışır. Bitince kütüphane ad sunucusuna, ad sunucusu da Reduce sunucusuna haber verir. Reduce sunucusu ara çiftleri kopyalar ve Reduce işleri başlar.
- **Gruplama (group by key):** Bir Reduce sunucusuna gelen çiftlerin anahtara göre gruplanması.
- **Düğüm arızalarının yönetimi.**
- **Makineler arası iletişim:** Map sunucusu, ad sunucusu ve Reduce sunucusu arasındaki iletişim.

3. Büyük Veri Yazılım Yığını

Hoca örnek bir büyük veri yığını (big data stack) gösterdi. Gösterilen katmanların hepsi açık kaynak kütüphanelerdi:

- **Dağıtık dosya sistemi:** Örnek HDFS. Bir dosya HDFS'e yüklenince HDFS onu parçalara böler ve HDFS sunucularının bulunduğu düğümlere dağıtır.
- **Kaynak yönetimi ve izleme:** Çok sayıda sunucuyu yönetmek için hangi sunucuların ayakta olduğunu ve sunucuların hangi adreslerde bulunduğunu gösteren bir araç gerekir. Mesos ve YARN bu amaçla HDFS sunucularını yönetmek ya da izlemek için kullanılan açık kaynak kütüphanelerdir.
- **Veri işleme araçları:** MapReduce programlama modelini gerçekleyen kütüphaneler. Hadoop ve Spark bunlardan ikisidir. Başka açık kaynak gerçeklemeler de vardır.
- **Uygulama:** En üstte programcının yazdığı uygulama yer alır. Uygulama Map ve Reduce fonksiyonlarını içerir. Kütüphane bu fonksiyonları dosya sistemi (HDFS) üzerinde çalıştırır.

  Hocanın vurgusu: Dönem projelerinde herkes dağıtık dosya sistemi olarak HDFS kullanacak.

4. Ana Düğüm ve Arıza Yönetimi

Hoca önceki derste anlatılan ana düğümün (master node, name node) görevini hatırlattı. Ana düğüm koordinasyonu üstlenir. Her Map ve Reduce görevinin durumunu izler: boşta (idle), sürüyor (in progress) ya da tamamlandı (completed).

Bir arıza olursa kütüphanenin yaptığı şey, arızalanan düğümün türüne bağlıdır.

4.1 Map İşçisinin Arızası

Bir Map sunucusu bozulursa, ana düğüm o sunucudaki Map görevlerinin durumuna bakar. Hocanın örneğinde arızalanan sunucuda üç Map görevi vardı. Durumu **tamamlandı** ya da **sürüyor** olan görevlerin durumu **boşta** yapılır. Böylece bu görevler başka bir Map sunucusunda yeniden çalıştırılır. Zaten boşta olan görevler de başka bir işçiye zamanlanır. Map görevlerini izlemek ad sunucusunun sorumluluğudur.

4.2 Reduce İşçisinin Arızası

Bir Reduce sunucusu bozulduğunda durum farklıdır. Hocanın örneğinde üç Reduce görevi vardı.

- Durumu **tamamlandı** olan Reduce görevi için hiçbir şey yapılmaz. Çünkü tamamlanmış bir Reduce görevinin sonucu dağıtık dosya sistemine (HDFS) yazılmıştır. Dağıtık dosya sistemine yazılan veri başka sunuculara da kopyalanır. Bu düğüm bozulsa bile sonucun kopyası başka bir sunucuda bulunur.
- Durumu **sürüyor** olan Reduce görevinin durumu **boşta** yapılır. Böylece yeniden zamanlanır ve tekrarlanır.

4.3 Ana Düğümün Arızası

Yalnız bir ana düğüm varsa ve o bozulursa bütün MapReduce görevi iptal edilir (aborted). Tüm MapReduce işleri baştan tekrarlanmak zorundadır.

4.4 Görev Sayıları

Hoca bir kez daha hatırlattı: 7 veri parçası varsa genelde 7 Map görevi olur. Reduce görevi sayısı genelde Map görevi sayısından azdır.

5. İyileştirme 1: Birleştirici (Combiner) Fonksiyonu

Hoca daha sonra iyileştirmelere (refinements) geçti. Bu bölümün sorusu şudur: MapReduce programlama modelini nasıl daha hızlı çalıştırabiliriz?

5.1 Sorun: Ara Çiftlerin Taşınması

MapReduce'ta en çok zaman alan iş, Map sunucularından Reduce sunucularına veri aktarımıdır. Map işleri bitip çıktılarını yerel depoya yazdıktan sonra ara çiftler Reduce sunucularına kopyalanmak zorundadır. Ara çift sayısı çok büyükse bu kopyalama çok uzun sürer.

Hocanın örneği: Terabaytlar mertebesinde bir metin dosyasında kelime sayılıyor olsun. Ara anahtar-değer çifti sayısı devasa olur. Bu kadar veriyi aynı veri merkezinde bile bir sunucudan diğerine taşımak çok zaman alır. Bu nedenle ara çiftlerin bir düğümden diğerine taşınma süresini azaltmanın bir yolu gerekir.

> **Ders sonrası not:** Hoca bu yeri anlatırken "map sunucuları ile data sunucuları arasındaki aktarım" dedi. Bağlamdan kastedilen, Map sunucuları ile Reduce sunucuları arasındaki aktarımdır.

5.2 Çözüm: Birleştirici

Çözüm birleştirici (combine) fonksiyonudur. Birleştirici, geliştiricinin yazdığı **üçüncü** fonksiyondur. Yani MapReduce programlarken Map, Reduce ve Combine fonksiyonları yazılır.

Birleştirici, Reduce fonksiyonuyla **aynı hesabı** yapar. Kelime sayma örneğinde Reduce toplama işlemiydi. O hâlde birleştirici de toplama işlemi olur. Fark, birleştiricinin bu hesabı **Map sunucusunda**, o sunucudaki ara çiftler üzerinde yapmasıdır. Böylece ara sonuçlar Reduce'a taşınırken bütün ara çiftler yerine yalnız anahtar ve o anahtarın değerlerinin toplamı kopyalanır.

5.3 Slayttaki Örnek

Hoca birleştiriciyi iyi gösteren bir örnek kullandı. İki veri parçası vardı ve her parçada çiftler bulunuyordu. Çiftlerin değerleri harflerdi (A, B, C, D, E gibi). Amaç, değer kısmında her harften kaç tane olduğunu bulmaktı. Yani kelime sayma örneğinin harflerle yapılan bir hâliydi. Map fonksiyonu her çift için (harf, 1) biçiminde bir ara çift üretiyordu.

- **Birleştirici olmadan:** Bir Map sunucusunda 6, diğerinde 6 ara çift vardı. Toplam 12 ara çift Reduce sunucusuna taşınmak zorundaydı.

- **Birleştirici ile:** Birleştirici, Map sunucusundaki ara çiftleri girdi olarak alır ve her harf için değerleri toplar.
  - Birinci sunucuda B, C, D ve E harfleri vardı. İki tane B olduğu için B'nin değeri 2, iki tane D olduğu için D'nin değeri 2 oldu.
  - İkinci sunucuda D, A, C ve B harfleri vardı. Birleştirici bu dört harfin değerlerini topladı.
  - Her sunucuda 4 çift kaldı. Toplam 8 çift taşındı.
- **Reduce tarafı:** Kütüphane gelen 8 çifti harflere göre grupladı. Örneğin B için birinci sunucudan 2, ikinci sunucudan 1 geldi. Gruplama sırasında hesap yapılmaz. Sonra Reducer, birleştirici gibi bu değerleri topladı ve B için son sayı olan 3'ü buldu.

12'den 8'e düşüş küçük görünebilir. Ama yüz binlerce ara çift olduğunda bu büyük bir iyileştirmedir. Birleştirici, bir sunucudan diğerine kopyalanması gereken veri miktarını azaltarak iletişim süresini kısaltır.

> **Ders sonrası not:** Hoca bu yeri özetlerken "iletişim süresini artırıyoruz" (increasing the communication time) dedi. Kastedilen, taşınan veri azaldığı için iletişim süresinin **azalmasıdır**.

Hoca birleştiriciyle ilgili soru olup olmadığını sordu, soru gelmedi. Ardından 10 dakikalık ara verildi.

6. Birleştiricinin Şartı: Değişme ve Birleşme Özellikleri

Aradan sonra hoca, birleştiricinin çalışabilmesi için bir şart olduğunu söyledi. Birleştirici olarak kullanılacak fonksiyonun değişme (commutative) ve birleşme (associative) özelliklerine sahip olması gerekir.

> **Ders sonrası not:** Hoca İngilizce anlatırken "communicative or associative" dedi. Doğru terim "commutative"tir (değişme özelliği). Ayrıca şart "veya" değil "ve"dir. Hocanın Türkçe anlatımında da iki özelliğin birlikte sağlanması istendi.

Hoca bunları matematikteki karşılıklarıyla açıkladı:

- **Birleşme özelliği:** Üç sayıyı toplarken bu sayıları kendi aralarında farklı gruplandırıp toplamak sonucu değiştirmiyorsa bu işlemde birleşme özelliği vardır: $(a + b) + c = a + (b + c)$.
- **Değişme özelliği:** İki sayıyı toplarken sıralarını değiştirmek sonucu değiştirmiyorsa bu işlemde değişme özelliği vardır: $a + b = b + a$.

Hoca bu örneği toplama için verdi. Aynı kontrol çarpma, bölme ve diğer fonksiyonlar için de, kullanılacak istatistiksel fonksiyon için de yapılmalıdır. Fonksiyon bu iki özelliği sağlıyorsa birleştirici eklenebilir, sağlamıyorsa eklenemez.

Bu özellikler aranan fonksiyon Reducer'daki fonksiyondur. Kelime sayma örneğinde Reducer toplama yapıyordu. Saymak aslında toplamadır. Toplama hem değişme hem birleşme özelliğine sahip olduğu için kelime saymaya birleştirici eklenebilir.

Slaytta farklı fonksiyonlar karşılaştırıldı:

- **Toplam (sum):** İki özelliği de sağlar. Birleştirici olarak kullanılabilir.
- **Ortalama (average):** Doğrudan kullanılamaz. Üç sayının ortalaması alınacaksa, önce birinci ile ikincinin ortalamasını alıp sonra bunu üçüncüyle ortalamak, ikinci ile üçüncüye öncelik verilen durumla aynı sonucu vermeyebilir.
- **Medyan (median):** Aynı şekilde doğrudan kullanılamaz.

  Ek bilgi (derste anlatılmadı): Ortalama, birleştirici ile büyük olasılıkla dolaylı yoldan hesaplanabilir. Birleştirici ortalama yerine (toplam, adet) çifti üretir. Reducer toplamları ve adetleri ayrı ayrı toplar, en sonda toplamı adede böler. Toplam ve adet işlemleri değişme ve birleşme özelliklerini sağladığı için sonuç gerçek ortalamaya eşit olur. Medyan için böyle basit bir ayrıştırma yoktur.

7. İyileştirme 2: Bölümleme (Partition) Fonksiyonu

Birleştirici her durumda eklenemiyordu. Yapılabilecek ikinci iyileştirme bölümleme fonksiyonudur. Burada amaç, anahtarların nasıl bölümleneceğini, yani hangi anahtar kümesinin tek bir Reduce işçisine gideceğini kontrol etmektir. Bunun için var olan bölümleme fonksiyonu programcının kendi fonksiyonuyla değiştirilir.

Normalde kütüphane, ara çiftleri Reducer'lara dağıtırken ara anahtarı bir hash fonksiyonuna sokar. Hash sonucunda hangi Reducer numarası çıkarsa çifti o Reducer'ın bulunduğu sunucuya gönderir. Hoca bunu birleştirici örneğine bağladı: B, C, D, E anahtarları bir hash fonksiyonuna girer ve çıkan numaraya göre Reducer seçilir. O örnekte tek Reducer olduğu için hash fonksiyonu hep aynı sunucuyu üretirdi. Birden fazla Reducer olsaydı her anahtar ilgili sunucuya giderdi.

MapReduce kütüphaneleri bu varsayılan fonksiyonun değiştirilmesine izin verir.

7.1 Örnek: Host Başına Toplam Sayfa Boyutu

Slayttaki problemde her host'a ait sayfaların toplam boyutu bulunmak isteniyordu. Amaç, aynı host adına (hostname) sahip bütün URL'lerin aynı Reducer'a gitmesiydi. O Reducer'da da aynı host adına sahip URL'lerin sayfa boyutları toplanacaktı.

- Varsayılan durumda hash fonksiyonuna anahtar olarak tam URL girerdi.
- Özel bölümleme fonksiyonunda önce URL'nin host adı bulunur, sonra o hash'lenir: `hash(hostname(URL)) mod R`. Burada R Reducer sayısıdır.

Böylece eşleme URL bazında değil, host adı bazında yapılır ve aynı host'a ait tüm URL'ler aynı Reducer'a gider.

> **Hocanın vurgusu:** Bölümleme fonksiyonunu değiştirmek MapReduce programını daha hızlı çalıştırmaz. Programcının kurduğu mantığa göre problemi daha etkili çözmesini sağlar.

8. MapReduce Gerçeklemeleri ve Bulut Ortamları

MapReduce programlama modelinin farklı gerçeklemeleri vardır. Hoca, büyük veri yığınını gösterirken bunlardan bahsettiğini hatırlattı:

- **Hadoop:** Açık kaynak gerçeklemelerden biri.
- **Google'ın kendi gerçeklemesi:** Google'ın da MapReduce'u gerçekleyen kendi kütüphanesi vardır.
- **Spark:** Slaytta görünmüyordu ama o da bir gerçeklemedir.
- **Hive ve Pig:** Arka planda Hadoop'un MapReduce'unu kullanan, SQL benzeri bir programlama arayüzü sunan açık kaynak kütüphanelerdir.

Bu kütüphanelerin hepsi Amazon ve Google bulutlarında kurulu hâlde bulunur. Hocanın hatırladığı kadarıyla bir öğrenci hesabı açıldığında yaklaşık 750 saatlik kullanım süresi veriliyor. Bu sürede bulutta MapReduce işleri çalıştırılabilir, ya da yerelde gerçeklenmiş bir MapReduce işi buluta yüklenip orada çalıştırılabilir.

> **Ek bilgi (derste anlatılmadı):** 750 saatlik süre büyük olasılıkla bulut sağlayıcılarının ücretsiz kullanım ya da öğrenci programlarındaki aylık kotalara karşılık gelir. Bu kotalar ve kapsadıkları servisler zamanla değişir, bu yüzden güncel koşullar sağlayıcının sayfasından kontrol edilmelidir.

9. Örnek Soru: Asal Bölenlerle MapReduce

3. bölüme geçmeden önce hoca MapReduce ile ilgili örnek bir soru çözdü.

Soru: Bir MapReduce işleminin girdisi tamsayılardan oluşuyor. Map fonksiyonu bir i tamsayısı alır ve i'yi bölen her asal p için (p, i) çiftini üretir. Örneğin $\text{map}(12) = [(2, 12), (3, 12)]$, çünkü 2 ve 3 hem asal hem de 12'yi bölüyor. Reduce fonksiyonu, kelime saymadaki gibi, her anahtarın değerlerini toplar. Girdi $\{15, 21, 24, 30, 49\}$ ise, verilen listedeki çiftlerden hangisi çıktıda bulunabilir?

Hoca, Reduce'un kelime saymadaki gibi bir toplama olduğunu, ama Map fonksiyonunun tanımına dikkat etmek gerektiğini söyledi.

Adım 1, Map çıktıları: Her girdi için hem onu bölen hem asal olan sayılar bulunur.

Girdi	Asal bölenler	Map çıktısı
15	3, 5	(3, 15), (5, 15)
21	3, 7	(3, 21), (7, 21)
24	2, 3	(2, 24), (3, 24)
30	2, 3, 5	(2, 30), (3, 30), (5, 30)
49	7	(7, 49)

Adım 2, gruplama: Kütüphane ara çiftleri anahtara göre gruplar. Anahtarlar 2, 3, 5 ve 7'dir. Bunlar zaten asal bölenlerdir.

- 2: 24, 30
- 3: 15, 21, 24, 30
- 5: 15, 30
- 7: 21, 49

Adım 3, Reduce: Her anahtarın değerleri toplanır.

- 2: $24 + 30 = 54$
- 3: $15 + 21 + 24 + 30 = 90$
- 5: $15 + 30 = 45$
- 7: $21 + 49 = 70$

Sonuç: Seçeneklerde çıktıda bulunan çift $(5, 45)$'tir. Hesap python ile doğrulandı.

> **Hocanın vurgusu:** Buna benzer soruları sınavda da görebilirsiniz. Dönem projesinde yapılacak programlama da buna benzer olacak: Verilen probleme göre Map ve Reduce fonksiyonlarının içinde ne olması gerektiğini belirleyecek, bunları gerçekleyecek, kullandığınız MapReduce kütüphanesine verecek ve kodu birden fazla sunucu üzerinde çalıştıracaksınız.

10. Benzer Öğeleri Bulma Problemi ve Uygulamaları

Hoca daha sonra 3. bölüme, benzer öğeleri bulmaya (Finding Similar Items, slaytta "finding similar sets") geçti. Önceki derslerde büyük veri kavramlarına giriş, MapReduce programlama modeli ve onu destekleyen yazılım yığını konuşulmuştu. Bu bölüm büyük veri algoritmalarının bir başka konusudur. Soru şudur: Tek bir bilgisayara sığmayan büyük ölçekli bir veri derleminde (data corpus) benzer veri kümelerini nasıl buluruz?

Hoca konuya uygulamalarla başladı.

10.1 Benzer Kelimeler İçeren Sayfalar

Benzer kelimeler içeren sayfaları bulup konularına göre sınıflandırmak istenebilir. Hocanın örneği: Size başlıksız 1000 Wikipedia sayfası verilsin, elinizde yalnız sayfaların içeriği olsun. Bunları spor, eğitim, siyaset gibi konulara ayırmanız istensin. Önce birbirine benzer sayfaları bulmanız gerekir. Benzer sayfalar bulunduktan sonra farklı konular altında sınıflandırılabilir.

10.2 Benzer Zevkteki Kullanıcılar

Netflix'te film zevki benzer kullanıcıları bulmak bir başka uygulamadır. Bir kullanıcı aksiyon, belgesel, klasik gibi farklı türlerde filmler izliyor olabilir. Benzer zevke sahip kullanıcılar bulunursa gruplanabilir. Böylece öneri yapılırken yalnız tek bir kişiye değil, aynı film zevkine sahip birçok kişiye birden öneri yapılabilir. Bu yüzden e-ticaret sitelerinde ve Netflix gibi yayın sitelerinde benzer zevkteki kullanıcıları bulmak çok önemlidir.

Tersi de yapılabilir: Benzer hayran kitlelerine (fans) sahip filmleri bulmak. Bu da öneri yapmayı sağlar.

10.3 Varlık Çözümleme

Varlık çözümleme (entity resolution) bir başka uygulamadır. Örneğin Facebook, Instagram ve Twitter'daki sosyal medya kayıtlarından aynı kişiye ait olanları bulmak istenebilir. Hoca burada "varlık" (entity) derken profili kastettiğini söyledi. Tek bir kişiye ait olabilecek benzer profil bilgilerini bulabilmek de yararlı bir bilgidir.

10.4 Ortak Metni Çok Olan Belgeler

Bir belge derlemi verildiğinde, ortak metni çok olan belge çiftlerini bulmak da bu konunun uygulamasıdır:

- **Ayna siteler (mirror sites):** Bir sitenin içeriğinin aynısını taşıyan kopya siteler olabilir. Aynı site birden fazla alan adında tutuluyor olabilir. Arama motorları bu siteleri tespit edip yalnız birini indekslemek ister.
- **İntihal (plagiarism):** Başka bir belgeden alınmış büyük alıntıları yakalamak.
- **Benzer haber makaleleri:** Birçok farklı sitede yayımlanmış, ortak metni çok olan benzer haber makalelerini bulmak.

11. Üç Adım: Shingling, MinHashing, LSH

Hocanın anlatacağı teknik üç temel adımdan oluşur:

- **Shingling:** Belgeyi kümeye dönüştürür. Belgede geçen karakter dizilerinin kümesini oluşturur.
- **MinHashing:** Belgeye ait kümelerin bir imzasını (signature) oluşturur.
- **Yerele duyarlı özetleme (Locality Sensitive Hashing, LSH):** İmzaları kullanarak aday çiftleri (candidate pairs), yani benzer olma olasılığı yüksek belge çiftlerini hesaplar.

Bu derste shingling, benzerlik ölçütleri ve MinHashing anlatıldı. LSH'nin bir sonraki derste ayrıntılı işleneceği söylendi.

12. Shingling

12.1 k-Shingle Tanımı

Bir k-shingle, doğal dil işlemede k-gram ya da n-gram olarak da bilinir. Bir belge için k-shingle, belgede geçen k karakterlik bir dizidir.

Oyuncak örnek: $k = 2$ ve belge $D_1 = abcab$ olsun. Beş harfli bir belgedir. Birinci karakterden başlayarak sırayla bütün ikili karakter dizileri yazılır:

- 1-2. karakterler: ab
- 2-3. karakterler: bc
- 3-4. karakterler: ca
- 4-5. karakterler: ab

Bu bir küme olduğu için aynı elemandan yalnız bir tane bulunabilir. ab daha önce yazıldığı için tekrar yazılmaz. Sonuç:

$$S(D_1) = \{ab, bc, ca\}$$

Yani belgeden iki karakterli üç dizi çıkarıldı ve belge bu kümeyle temsil edildi. Sonuç python ile doğrulandı.

12.2 Neden Shingle?

Amaç, iki belgenin birbirine ne kadar benzediğini belirli bir yönteme dayanarak söylemektir. Sezgisel olarak, iki belgede çok sayıda ortak kelime varsa bunlar yüksek oranda benzer belgelerdir. Shingling aynı mantığı kelimeler yerine k-shingle'larla uygular: Sezgisel olarak benzer belgelerin çok sayıda ortak shingle'ı olur.

Önemli bir özellik: Bir kelimeyi değiştirmek yalnız o kelimeye yakın shingle'ları etkiler.

12.3 Kelime Değiştirme Örneği

Slayttaki örnekte $k = 3$ seçildi ve iki tek cümlelik belge vardı:

- D_1 = “The dog which chased the cat”
- D_2 = “The dog that chased the cat”

Her cümle birinci karakterden başlanarak üçlülere dönüştürülür: `The`, `he`, `e d`, `do` diye devam eder. İki cümle aslında aynıdır, yalnız “which” yerine “that” konmuştur.

Hoca bu değişiklikle D_1’deki 7 shingle’ın değiştiğini saydı: `g w`, `wh`, `whi`, `hic`, `ich`, `ch`, `h c`. Bunlar yerine D_2’de başka shingle’lar gelir. Bunların dışındaki bütün 3-shingle’lar iki belgede aynı kalır.

Python ile doğrulama: D_1’in 25, D_2’nin 23 farklı 3-shingle’ı vardır. D_1’de olup D_2’de olmayan shingle’lar tam olarak yukarıdaki 7 tanedir. D_2’de olup D_1’de olmayanlar 5 tanedir: `g t`, `tha`, `hat`, `at`, `t c`. Ortak 18 shingle vardır. İki kümenin birleşimi 30 elemanlıdır.

> **Ders sonrası not:** Hoca, değişen kelimenin “2K karakter öncesi ve 2K karakter sonrasına kadar” olan shingle’ların etkilendiğini söyledi. Doğrusu: Etkilenen shingle’lar, değişen karakterlerle en az bir karakter paylaşan shingle’lardır. Bunlar değişen kısmın en fazla $k-1$ karakter öncesinden başlar ve en fazla $k-1$ karakter sonrasında biter. Örnekte $k = 3$ olduğu için “which” kelimesinden önce 2 ve sonra 2 karakter etkilenir. Slayttaki 7 shingle bununla tutarlıdır.

12.4 Shingle’ları Tamsayıya Dönüştürmek

Shingle’lar bellekte daha az yer tutsun isteniyorsa tamsayılara dönüştürülür. Her shingle’ı bir tamsayıya hash’leyen bir hash fonksiyonu yazılır. Böylece shingle’lar örneğin 4 bayt ile gösterilir.

Yani kümedeki elemanlar artık `g w`, `wh`, `whi` gibi karakter dizileri olmaz. Bütün shingle’lar tespit edilip bir shingle uzayı oluşturulduktan sonra, bu karakter dizileri hash fonksiyonuyla tamsayılara eşlenir. Bellekte karakter dizileri yerine onlara karşılık gelen sayılar tutulur. Her belgenin shingle kümesi sayılardan oluşan bir kümeyle temsil edilir. İki belge karşılaştırılırken iki belgede ortak olan bu sayısal değerler ne kadar fazlaysa belgeler o kadar benzerdir.

13. Jaccard Benzerliği

İki belge için k-shingle kümeleri oluşturulduktan sonra benzerliklerinin ölçülmesi gerekir. Kullanılabilecek yöntemlerden biri Jaccard benzerliğidir (Jaccard similarity).

Jaccard benzerliği şunu söyler: Elinde iki küme varsa, önce kesişimlerini bul ve eleman sayısını al. Bunu birleşimlerinin eleman sayısına böl.

$$\text{sim}(C_1, C_2) = \frac{|C_1 \cap C_2|}{|C_1 \cup C_2|}$$

Bu oran yüksekse kümeler birbirine daha çok, düşükse daha az benzer.

Slayttaki örnek: İki kümenin kesişiminde 3 eleman, birleşiminde 8 eleman vardı. Jaccard benzerliği:

$$\text{sim} = \frac{3}{8} = 0{,}375$$

14. Belgeleri 0/1 Kolon Vektörü Olarak Göstermek

14.1 Evrensel Küme ve Kolon Vektörleri

Bir belge kümelerle tanımlanabiliyorsa şöyle düşünülebilir: Belgelerden çıkarılan bütün k-shingle’lar tek bir uzayda toplanır. Hocanın örneği: Birinci belgeden 5, ikinci belgeden 3 k-shingle çıkmış olsun. Bunların hiç ortak elemanı yoksa toplam 8 farklı karakter dizisi olur.

Uzayda 8 eleman vardır. O zaman birinci belge 8 boyutlu bir vektörle gösterilebilir: Birinci shingle'dan başlayarak, shingle belgede varsa 1, yoksa 0 yazılır.

Genel yöntem:

- Belgelerden çıkarılan bütün shingle'lar bir evrensel küme (universal set) olarak düşünülür. Her shingle bir satırdır.
- Her belge bir kolon vektörü olarak gösterilir.
- Her satırda, o satırın shingle'ı belgede varsa 1, yoksa 0 yazılır.

Vektörün boyutu, yani matrisin satır sayısı, bütün belgelerden çıkarılan farklı (unique) k-shingle sayısıdır. Bu, bir belgenin alabileceği en büyük boyuttur. Belgeler böyle gösterilince Jaccard benzerliğini hesaplamak çok kolaylaşır.

14.2 Slayttaki Örnek

Slayttaki örnekte iki belgeden çıkarılan farklı k-shingle sayısı 6'ydı. Uzay birinciden altıncıya kadar 6 satırdan oluşuyordu. C_1 ve C_2 belgelerinin her biri, shingle'ın belgede olup olmamasına göre 0/1 kolon vektörü olarak yazıldı.

Jaccard benzerliği için:

- Pay: İki kolonda da 1 olan satırların sayısı. Örnekte 2.
- Payda: En az birinde 1 olan satırların sayısı. Örnekte 5.

$$\text{sim}(C_1, C_2) = \frac{2}{5} = 0{,}4$$

14.3 Satır Türleri ile Formül

Hoca bu hesabı formülleştirdi. İki kolonun bir satırı dört türden biridir:

- **Tür a:** İki kolonda da 1.
- **Tür b:** C_1'de 1, C_2'de 0.
- **Tür c:** C_1'de 0, C_2'de 1.
- **Tür d:** İki kolonda da 0. Benzerliğe katılmaz.

a, b ve c türlerinin hepsinde kolonlardan en az birinde 1 vardır. Bu yüzden $a+b+c$ birleşimin eleman sayısıdır ve Jaccard benzerliği şöyle yazılır:

$$\text{sim}(C_1, C_2) = \frac{a}{a+b+c}$$

14.4 Ölçek Sorunu

Bu kadar küçük bir oyuncak örnekte benzerliği hesaplamak kolaydır. Ama satır sayısı 100.000'ler, belge sayısı da 100.000'ler mertebesindeyse problem zorlaşır. Hem belge sayısı hem de her belgenin satır sayısı çok büyükse iki belge arasındaki benzerlik nasıl bulunur? Bunun için MinHashing yöntemi kullanılır.

15. MinHashing ve İmza Matrisi

Shingling yöntemi belgelerden kümeler oluşturur ve her belgeyi 0 ve 1'lerden oluşan bir kolon vektörüyle gösterir. MinHashing ise bu belgeleri bir imza matrisiyle (signature matrix) temsil etmeyi sağlar.

15.1 Girdi Matrisi

Hoca örneği şu matris üzerinden anlattı. Dört belge vardı: C_1, C_2, C_3, C_4. Bütün belgelerde olabilecek 7 farklı eleman (shingle) vardı. Her belge için her elemanın belgede olup olmamasına göre 1 ya da 0 yazılmıştı. Örneğin C_1'de birinci ve ikinci eleman vardı. Üçüncü, dördüncü ve beşinci eleman yoktu. Altıncı ve yedinci eleman vardı.

Satır	C1	C2	C3	C4
1	1	0	1	0
2	1	0	0	1
3	0	1	0	1
4	0	1	0	1
5	0	1	0	1
6	1	0	1	0
7	1	0	1	0

Elimizde her biri 7 uzunluğunda dört belge vardır. Problem, bunlardan birbirine benzer olanları bulmaktır. Örnek bu kadar küçükken Jaccard benzerliğiyle her çifti denemek kolaydır: C_1-C_2, C_1-C_3, C_1-C_4, C_2-C_3 ve diğerleri. Ama belge sayısı yüz binler ya da bir milyon mertebesindeyse olası çift sayısı inanılmaz boyutlara ulaşır. Satır sayısı da 7'den yüz binlere, milyonlara çıkarsa benzerliği bulmak hem çok zaman alır hem hesaplama karmaşıklaşır.

Bu yüzden belgeleri, belgeden **bilgi kaybetmeden** daha küçük bir biçimde temsil etmek gerekir. Hedef, örneğin birinci belgeyi 7 satır yerine 3 satırdan oluşan bir kolon vektörüyle göstermektir. Bu vektörlerden oluşan matris imza matrisidir.

15.2 Hash Fonksiyonu: Permütasyon ve İlk 1

Girdi matrisinde kolonlar belgeleri, satırlar belgelerin içerdiği elemanları gösterir. Bu matristen imza matrisine geçerken hash fonksiyonları kullanılır. Örnekte 3 farklı hash fonksiyonu vardı.

Her hash fonksiyonu girdi olarak bir kolon vektörü alır ve **tek bir sayısal değer** üretir. Hash fonksiyonunun içinde satırların rastgele bir sıralaması, yani bir permütasyonu vardır. 1'den 7'ye kadar sayılar satırlara rastgele atanmıştır. Fonksiyon bu sıralamada 1'den başlayarak ilerler. Kolonun ilk kez 1 değerini aldığı satırın sıra numarası fonksiyonun çıktısıdır.

Hocanın İngilizce tanımı: Permütasyon sırasında, C kolonunun 1 değerini aldığı ilk satırın numarası hash fonksiyonunun sonucudur.

$$h_\pi(C) = \min_{r\,:\,C[r]=1} \pi(r)$$

Slayttaki permütasyonlar ($\pi(r)$, r. satırın permütasyondaki sırası):

Satır	π1	π2	π3
1	3	4	1
2	4	2	3
3	7	1	7
4	6	3	6
5	1	6	2
6	2	7	5
7	5	5	4

15.3 Birinci Hash Fonksiyonu (π_1)

Hoca birinci hash fonksiyonunu her belge için adım adım uyguladı.

C_1 **için:** Sıra 1'deki satır 5. satırdır. C_1'de bu satır 0'dır, devam edilir. Sıra 2'deki satır 6. satırdır. C_1'de bu satır 1'dir. İlk 1, sıra 2'de bulundu. Sonuç **2**.

C_2 **için:** Sıra 1'deki satırda (5. satır) C_2 zaten 1'dir. Sonuç **1**.

C_3 **için:** Sıra 1'de 0, sıra 2'de 1. Sonuç **2**.

C_4 **için:** Sıra 1'de 1. Sonuç **1**.

Böylece birinci hash fonksiyonu için dört belgenin değerleri tek birer sayı olarak bulundu: $2, 1, 2, 1$.

15.4 İkinci ve Üçüncü Hash Fonksiyonları

İkinci hash fonksiyonu farklı bir sıralama kullanır.

π_2**,** C_1 **için:** Sıra 1'deki satır 3. satırdır, C_1'de 0. Sıra 2'deki satır 2. satırdır, C_1'de 1. Sonuç **2**. Bu değer imza matrisinde ikinci hash fonksiyonunun satırına, birinci belgenin kolonuna yazılır.

π_2**,** C_3 **için:** Sıra 1'deki satır (3. satır) 0, sıra 2'deki satır (2. satır) 0, sıra 3'teki satır (4. satır) 0, sıra 4'teki satır (1. satır) 1. Sonuç **4**.

π_3**,** C_4 **için:** Sıra 1'deki satır 1. satırdır, C_4'te 0. Sıra 2'deki satır 5. satırdır, C_4'te 1. Sonuç **2**.

Tüm hesaplar yapıldığında imza matrisi M şöyle olur (python ile doğrulandı):

	C1	C2	C3	C4
π1	2	1	2	1
π2	2	1	4	1
π3	1	2	1	2

15.5 İmza Matrisinin Boyutu

İmza matrisinde her hash fonksiyonu için bir satır, her belge için bir kolon vardır. Hoca bunu sorularla pekiştirdi: Dört hash fonksiyonu olsaydı dört satır, beş hash fonksiyonu olsaydı beş satır olurdu. Dört yerine altı belge olsaydı altı kolon olurdu.

Böylece dört belgeden oluşan girdi matrisi, yine dört kolonlu ama satır sayısı çok daha az olan bir imza matrisine dönüştürüldü. MinHashing budur.

Bu işlemin amacı, konunun ana teması olan belgeler arası benzerliği bulmaktır. Her belgenin satır sayısı milyonlar mertebesindeyse iki belge arasında Jaccard benzerliğini hesaplamak çok uzun sürer. Bundan kaçınmak için her belge, bilgi kaybı olmadan daha az satırlı bir kolon vektörüyle temsil edilir. Satır sayısı hash fonksiyonu sayısı kadardır, kolon sayısı belge sayısı kadar kalır.

> **Ders sonrası not:** Hoca "MinHashing" adının birden fazla hash fonksiyonu kullanılmasından geldiğini söyledi. Ad aslında her hash fonksiyonunda permütasyondaki **en küçük** (minimum) sıranın alınmasından gelir: $h_\pi(C) = \min \pi(r)$, burada r kolonun 1 olduğu satırlardır. Birden fazla hash fonksiyonu kullanmak imzanın uzunluğunu belirler.

16. İmza Benzerliği ile Jaccard Benzerliğinin Karşılaştırılması

16.1 Temel Özellik

Hoca şunu söyledi: Rastgele bir permütasyonda iki kolonun MinHash değerlerinin eşit olma olasılığı, iki kolonun Jaccard benzerliğine eşittir.

$$P[h_\pi(C_1) = h_\pi(C_2)] = \text{sim}(C_1, C_2)$$

Yani C_1 ve C_2 belgelerinin girdi matrisindeki Jaccard benzerliği ile imza matrisindeki iki kolonun benzerliği neredeyse aynıdır. Bu nedenle imza matrisi, benzerlik açısından girdi matrisini yansıtır.

> **Ders sonrası not:** Hoca İngilizce anlatımda bunu "iki belgenin aynı olma olasılığı, imza matrisinde iki kolonun aynı olma olasılığına eşittir" diye ifade etti. Doğru hâli yukarıdaki formüldür: Tek bir rastgele permütasyon için MinHash değerlerinin eşit olma olasılığı Jaccard benzerliğine eşittir.
>
> **Ek bilgi (derste anlatılmadı):** Bu özelliğin sebebi büyük olasılıkla kitaptaki şu akıl yürütmedir. İki kolonda da 0 olan (d türü) satırlar sonucu etkilemez. Permütasyonda a, b veya c türünden ilk karşılaşılan satıra bakılır. Bu satır a türündeyse iki kolonun MinHash değeri eşit olur, b veya c türündeyse farklı olur. Permütasyon rastgele olduğu için bu ilk satırın a türünden olma olasılığı $a/(a+b+c)$'dir, yani tam olarak Jaccard benzerliğidir.

16.2 İmza Benzerliğinin Tanımı

İki imzanın benzerliği, iki imza kolonunun **aynı değeri aldığı** hash fonksiyonlarının oranıdır. Hoca bunu "iki kolonun hemfikir olduğu satırların yüzdesi" diye anlattı. İki imzanın beklenen benzerliği, imzaların temsil ettiği kolonların Jaccard benzerliğine eşittir.

16.3 Slayttaki Karşılaştırma

Çift	1-3	2-4	1-2	3-4
Kolon/Kolon (Jaccard)	0,75	0,75	0	0
İmza/İmza	0,67	1,00	0	0

Hoca çiftleri tek tek inceledi:

- **2 ile 4:**
  - İmza matrisinde $C_2 = (1, 1, 2)$ ve $C_4 = (1, 1, 2)$ tamamen aynıdır. İmza benzerliği 1.
  - Jaccard için girdi matrisinde iki kolonda da 1 olan satırlar 3, 4 ve 5. satırlardır, yani 3 satır. En az birinde 1 olan satırlar 2, 3, 4 ve 5. satırlardır, yani 4 satır. $3/4 = 0{,}75$.
- **1 ile 2:** İmza matrisinde hiçbir satır eşleşmiyor, imza benzerliği 0. Girdi matrisinde iki kolonda da 1 olan satır yok, Jaccard benzerliği de 0.
- **1 ile 3:**
  - İmza matrisinde $C_1 = (2, 2, 1)$ ve $C_3 = (2, 4, 1)$. Birinci ve üçüncü satır eşleşiyor, ikinci satır eşleşmiyor. İmza benzerliği $2/3 \approx 0{,}67$.
  - Jaccard benzerliği $0{,}75$. İki kolonda da 1 olan satırlar 1, 6 ve 7. satırlardır. En az birinde 1 olanlar 1, 2, 6 ve 7. satırlardır. Sonuç $3/4$ (ara adım eklendi).
- **3 ile 4:** İki benzerlik de 0.

Tablodaki tüm değerler python ile doğrulandı.

Bu karşılaştırma şunu gösterir: İmza matrisindeki iki belge arasındaki benzerlik, imza matrisi hiç kullanılmadan girdi matrisi üzerinden hesaplanan benzerliğe yakınsar. Aradaki fark çok büyük değildir. Hash fonksiyonu sayısı artırılırsa değerler birbirine daha da yaklaşır: 0,75 ile 0,67 arasındaki fark da, 0,75 ile 1 arasındaki fark da azalır.

> **Hocanın vurgusu:** Hash fonksiyonu sayısı artarsa imza matrisinin satır sayısı artar ve imza benzerliği, girdi matrisindeki benzerliği daha iyi yakalar.

Hoca 3. bölümün 12. slaytında durdu. Bir sonraki derste MinHashing'in gerçeklenmesine (implementation) ve ardından LSH'nin ayrıntılarına geçileceğini söyledi.

17. Soru-Cevap: Girdi Matrisinden İmza Matrisine Geçiş

Ders sonunda bir öğrenci, girdi matrisinden imza matrisine nasıl geçildiğini kaçırdığını söyledi ve yeniden anlatılmasını istedi. Hoca konuyu baştan, adım adım tekrar anlattı.

Soru: Girdi matrisinden imza matrisine nasıl geçiliyor?

Cevap: Hoca önce girdi matrisindeki 1 ve 0'ların nereden geldiğini açıkladı:

- Amaç belgeler arası benzerliği bulmaktır.
- Her belgedeki bütün karakter dizileri çıkarılır ve bir kümeye dönüştürülür. Her karakter dizisine shingle denir.
- Dört belgenin her birinden diziler çıkarılır ve bir araya getirilir. Örneğin her eleman iki karakterli olsun. Dört belgeden çıkabilecek bütün farklı iki karakterli elemanlar bulunur. Diyelim ki 7 tane çıktı: "ab", "bc", "cd", "de" gibi.
- Bunlar yukarıdan aşağıya yazılır. Her belge, bu elemanları içerip içermemesine göre 0/1 ile temsil edilir. Birinci iki karakterli eleman C_1'de varsa 1, yoksa 0 yazılır. Böylece C_1 bir kolon vektörü olur. C_2, C_3 ve C_4 de aynı şekilde tanımlanır.

Sonra benzerliğe geçti. Jaccard benzerliğinde iki kolonda da 1 olan satır sayısı paya, en az birinde 1 olan satır sayısı paydaya yazılır. C_1 ile C_2'de iki kolonda da 1 olan satır olmadığı için pay sıfırdır. Payda ne olursa olsun benzerlik sıfır çıkar. Sorun şudur: 7 değil, milyon ya da

milyar satırlı iki belge karşılaştırılırsa Jaccard benzerliği için harcanacak zaman çok fazla olur. Bu yüzden iki kolonu karşılaştırmanın başka bir yolu gerekir. MinHashing bu yoldur.

Hoca hash fonksiyonlarını yeniden tanımladı. Her hash fonksiyonunun içinde 1'den 7'ye kadar sayıların bir permütasyonu vardır. Permütasyon bulunduktan sonra, sıralamada birinciye karşılık gelen satırda belgenin değeri 1 ise hash fonksiyonunun sonucu 1 alınır. Değilse bir sonrakine geçilir. Sıra 2'ye karşılık gelen değer 1 ise sonuç 2 alınır, böyle devam edilir. Yani permütasyon sırasında belgenin değerinin 1 olduğu ilk sıra hash fonksiyonunun sonucudur. Birinci hash fonksiyonu ve birinci belge için: Sıra 1'e bakıldı, 0. Sıra 2'ye bakıldı, 1. Sonuç 2, imza matrisinin ilk satırına birinci belge için yazılır.

Ardından hoca öğrenciye iki hesap yaptırdı. Bunlar aşağıdaki "Örnek Sorular" bölümünde yer alıyor.

Hoca anlatımı şöyle bağladı. Her hash fonksiyonu için bir satır ayrılır. Hash fonksiyonunun girdisi belgenin kendisi, yani kolon vektörüdür. Çıktısı tek bir sayısal değerdir. Birinci kolon vektörü verilince bir değer, ikinci verilince bir değer üretilir ve bu dört belge için tekrarlanır. Bu birinci hash fonksiyonu içindir. Aynısı ikinci ve üçüncü için yapılır. Dördüncü, beşinci, altıncı hash fonksiyonu olsaydı satır sayısı artardı.

Kazanç şudur: Satır sayısı milyar mertebesinde olunca iki belge arasındaki Jaccard benzerliğini bulmak çok uzun sürer. İmza matrisinde ise iki belge karşılaştırılırken yalnız üç boyutlu iki vektör karşılaştırılır. Hesaplama süresi birden çok düşer. Peki bu imza matrisi benzerliği gerçekten yakalıyor mu? Örnek oyuncak olduğu için 3 hash fonksiyonu kullanıldı. Hash fonksiyonu sayısı örneğin 100 olursa, girdi matrisinde satır sayısı ne kadar fazla olursa olsun, imza matrisindeki iki kolonun benzerliği girdi matrisindeki iki kolonun benzerliğine çok yakınsar. Böylece imza matrisi üzerinden benzerlik hesaplanınca girdi matrisindeki benzerlik yakalanmış olur. Bu, satır sayısı fazla olan büyük veri setlerinde iki kolon arasındaki benzerliği daha hızlı bulmayı sağlar.

Öğrenci anladığını söyledi.

Örnek Sorular

Bu dersin videosu ve karesi yoktur. Aşağıdaki sorular derste sözlü sorulan ya da hocanın sesli okuduğu sorulardır.

Soru 1: Asal Bölenlerle MapReduce (derste sözlü, 01:16:40)

Soru: Bir MapReduce işleminin girdisi tamsayılardır. Map fonksiyonu bir i tamsayısı alır ve i'yi bölen her asal p için (p, i) çiftini üretir. Örneğin map$(12) = [(2, 12), (3, 12)]$. Reduce fonksiyonu her anahtarın değerlerini toplar. Girdi $\{15, 21, 24, 30, 49\}$ ise, listedeki çiftlerden hangisi çıktıda bulunabilir?

Çözüm (derste verildi): Map çıktıları anahtara göre gruplanınca 2: {24, 30}, 3: {15, 21, 24, 30}, 5: {15, 30}, 7: {21, 49} elde edilir. Reduce toplamları: (2, 54), (3, 90), (5, 45), (7, 70). Seçeneklerde bulunan çift **(5, 45)**'tir. Ayrıntılı çözüm 9. bölümdedir.

Soru 2: Birleştirici Özelliği Hangi Fonksiyonda Aranır? (derste sözlü, 01:07:11)

Soru: Değişme ve birleşme özellikleri hangi fonksiyonda aranır?

Çözüm (derste verildi): Reducer'daki fonksiyonda. Kelime sayma örneğinde Reducer toplama yapar, toplama iki özelliği de sağlar. Bu yüzden birleştirici eklenebilir.

Soru 3: Ortalama Birleştirici Olabilir mi? (derste sözlü, 01:07:56)

Soru: Reduce fonksiyonu ortalama (average) ise birleştirici kullanılabilir mi?

Çözüm (derste verildi): Hayır. Sayılar farklı gruplandırılıp ortalamaları alınırsa, örneğin önce birinci ile ikincinin ortalaması alınıp sonra üçüncüyle ortalanırsa, sonuç ikinci ile üçüncüye öncelik verilen durumla aynı çıkmayabilir. Medyan için de durum aynıdır.

Soru 4: İmza Matrisinin Boyutu (derste sözlü, 02:01:24)

Soru: Dört yerine altı belge olsaydı imza matrisinde kaç kolon olurdu? Dört ya da beş hash fonksiyonu olsaydı kaç satır olurdu?

Çözüm (derste verildi): Altı belge için altı kolon olur. Satır sayısı hash fonksiyonu sayısına eşittir: dört hash fonksiyonu için dört satır, beş için beş satır.

Soru 5: π_1 ile C_2'nin MinHash Değeri (derste sözlü, 02:18:07)

Soru: Birinci hash fonksiyonunun permütasyonu verilmişken ikinci belge (C_2) için hash değeri nedir ve imza matrisinde nereye yazılır?

Çözüm (derste verildi, öğrenci cevapladı): Sıra 1'deki satırda (5. satır) C_2'nin değeri 1'dir. İlk 1 hemen görüldüğü için sonuç **1**. Bu değer imza matrisinde birinci hash fonksiyonunun satırına, ikinci belgenin kolonuna yazılır.

Soru 6: π_3 ile C_2'nin MinHash Değeri (derste sözlü, 02:18:43)

Soru: Üçüncü hash fonksiyonu için ikinci belgenin (C_2) hash değeri nedir?

Çözüm (derste verildi, öğrenci cevapladı): Sıra 1'deki satırda (1. satır) C_2'nin değeri 0'dır, devam edilir. Sıra 2'deki satırda (5. satır) değer 1'dir. İlk 1'in görüldüğü sıra 2 olduğu için sonuç **2**. Python ile doğrulandı.

Dersin Toparlaması

Dersin ilk yarısında MapReduce programlama modeli tekrar edildi. Programcı yalnız Map ve Reduce fonksiyonlarını yazar. Girdinin bölünmesi, zamanlama, anahtara göre gruplama, arıza yönetimi ve makineler arası iletişim kütüphanenin işidir. Map işçisi bozulursa tamamlanmış ve süren Map görevleri tekrarlanır. Reduce işçisi bozulursa yalnız süren görevler tekrarlanır, çünkü tamamlananların çıktısı HDFS'tedir. Ana düğüm bozulursa bütün iş iptal edilir. Ardından iki iyileştirme görüldü. Birleştirici, Reduce ile aynı hesabı Map sunucusunda yaparak ağ üzerinden taşınan ara çift sayısını azaltır. Ama yalnız Reduce fonksiyonu değişme ve birleşme özelliklerine sahipse kullanılabilir: toplama uygundur, ortalama ve medyan doğrudan uygun değildir. Bölümleme fonksiyonunu değiştirmek programı hızlandırmaz, problemin mantığına uygun eşleme yapmayı sağlar. Asal bölenler sorusu, sınavda ve dönem projesinde beklenen düşünme biçimini gösterdi.

Dersin ikinci yarısında 3. bölüme, benzer öğeleri bulmaya geçildi. Belgeler önce shingling ile k karakterlik dizilerin kümesine dönüştürülür. İki küme arasındaki benzerlik Jaccard benzerliği ile ölçülür. Belgeler 0/1 kolon vektörleri olarak yazılınca bu benzerlik $a/(a+b+c)$ olur. Satır ve belge sayısı çok büyük olduğunda MinHashing devreye girer: Her permütasyon için kolonun ilk 1 değerini aldığı sıra alınır ve küçük bir imza matrisi oluşur. İki imza kolonunun eşleşme oranı Jaccard benzerliğine yaklaşır, hash fonksiyonu sayısı arttıkça da yaklaşım iyileşir. Bir sonraki derste 3. bölümün 12. slaytından, MinHashing'in gerçeklenmesinden devam edilecek ve ardından yerele duyarlı özetleme (LSH) işlenecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.